Solution: Smallest Subarray With a Greater Sum

Problem Statement

Given an array of positive integers and a number ‘S,’ find the length of the **smallest** contiguous subarray whose sum is **greater than or equal to 'S'**. Return 0 if no such subarray exists.

Example 1:

```
Input: [2, 1, 5, 2, 3, 2], S=7
Output: 2
Explanation: The smallest subarray with a sum greater than or equal to
'7' is [5, 2].
```

Example 2:

```
Input: [2, 1, 5, 2, 8], S=7
Output: 1
Explanation: The smallest subarray with a sum greater than or equal to
'7' is [8].
```

Example 3:

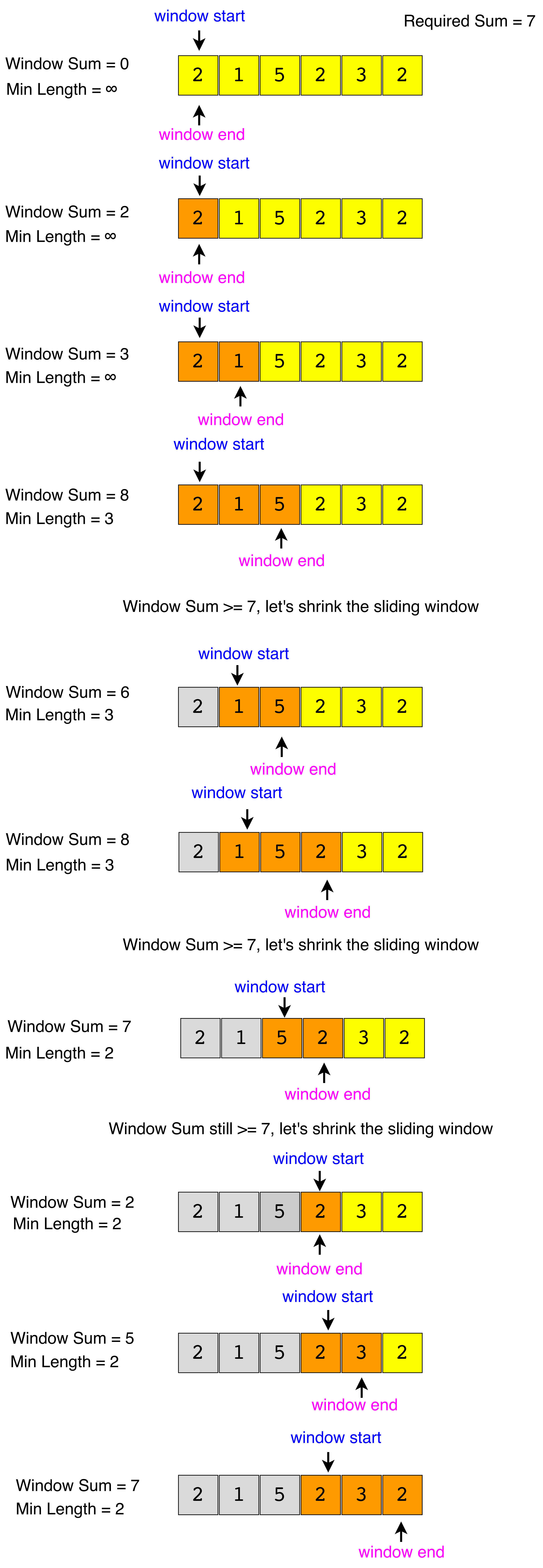
```
Input: [3, 4, 1, 1, 6], S=8
Output: 3
Explanation: Smallest subarrays with a sum greater than or equal to '8'
are [3, 4, 1] or [1, 1, 6].
```

Solution

This problem follows the **Sliding Window** pattern, and we can use a similar strategy as discussed in the Maximum Sum Subarray of Size K. There is one difference though: in this problem, the sliding window size is not fixed. Here is how we will solve this problem:

1. First, we will add up elements from the beginning of the array until their sum becomes greater than or equal to 'S.'
2. These elements will constitute our sliding window. We are asked to find the smallest such window having a sum greater than or equal to 'S.' We will remember the length of this window as the smallest window so far.
3. After this, we will keep adding one element in the sliding window (i.e., slide the window ahead) in a stepwise fashion.
4. In each step, we will also try to shrink the window from the beginning. We will shrink the window until the window's sum is smaller than 'S' again. This is needed as we intend to find the smallest window. This shrinking will also happen in multiple steps; in each step, we will do two things:
    - Check if the current window length is the smallest so far, and if so, remember its length.
    - Subtract the first element of the window from the running sum to shrink the sliding window.

Here is the visual representation of this algorithm for the Example-1:



Code

Here is what our algorithm will look like:

Java

```
public class Solution {
    public int findMinSubArray(int S, int[] arr) {
        int windowSum = 0, minLength = Integer.MAX_VALUE;
        int windowStart = 0;
        for (int windowEnd = 0; windowEnd < arr.length; windowEnd++) {
            windowSum += arr[windowEnd]; // add the next element
            // shrink the window as small as possible until the 'windowSum' is smaller t
            while (windowSum >= S) {
                minLength = Math.min(minLength, windowEnd - windowStart + 1);
                windowSum -= arr[windowStart]; // subtract the element going out
                windowStart++; // slide the window ahead
            }
        }

        return minLength == Integer.MAX_VALUE ? 0 : minLength;
    }

    public static void main(String[] args) {
        Solution sol = new Solution();
        int result = sol.findMinSubArray(7, new int[] { 2, 1, 5, 2, 3, 2 });
        System.out.println("Smallest subarray length: " + result);
```

Run Code Result Stdout

You must run your code first

Execute

Time Complexity

The time complexity of the above algorithm will be $O(N)$. The outer `for` loop runs for all elements, and the inner `while` loop processes each element only once; therefore, the time complexity of the algorithm will be $O(N+N)$, which is asymptotically equivalent to $O(N)$.

Space Complexity

The algorithm runs in constant space $O(1)$.

← Previous
Smallest Subarray With A Greater Sum (Easy)

Next →
Longest Substring With K Distinct Characters (Medium)

☐ Mark as Completed